

TUTORIAL 2

Sai Keerthana Karnam

IIT Kharagpur

February 18, 2026



Question 1: Attention

Scaled dot-product attention: Given the following matrices for query Q , key K , and value V :

$$Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

with $d_k = 3$, compute the output of the scaled dot-product attention mechanism as follows:

- (a) Compute the attention scores by taking the dot product between Q and K^T , then divide by $\sqrt{d_k}$.
- (b) Apply the softmax function to the attention scores to normalize them.
- (c) Compute the weighted sum of values V using the attention scores obtained from (b).

Question 1: Attention (solution)

(a) QK^T

$$= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\frac{QK^T}{\sqrt{dk}} = \begin{bmatrix} 1/\sqrt{3} & 1/\sqrt{3} \\ 1/\sqrt{3} & 1/\sqrt{3} \end{bmatrix}$$

(b)

$$\begin{bmatrix} \frac{e^{1/\sqrt{3}}}{e^{1/\sqrt{3}} + e^{1/\sqrt{3}}} & \frac{e^{1/\sqrt{3}}}{e^{1/\sqrt{3}} + e^{1/\sqrt{3}}} \\ \frac{e^{1/\sqrt{3}}}{e^{1/\sqrt{3}} + e^{1/\sqrt{3}}} & \frac{e^{1/\sqrt{3}}}{e^{1/\sqrt{3}} + e^{1/\sqrt{3}}} \end{bmatrix} = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}$$

(c)

$$\begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 3 \\ 2 & 3 \end{bmatrix}$$

Question 2: Counting number of parameters

Suppose you are training a Transformer encoder–decoder model for the task of text summarization. Assume that the source vocabulary is 60k. Across encoder and decoder, you are using model dimensionality $d_{\text{model}} = 1024$ and the feed-forward hidden dimension is $d_{\text{ff}} = 2048$. Assume that you use **multi-head attention with $h = 16$ heads** in all attention modules. Also assume that both encoder and decoder use the **same** positional embeddings, and the maximum sequence length is 512. The encoder uses 24 layers and the decoder uses 12 layers. Assume that there is weight tying for the *unembedding*

- (a) Calculate the total number of trainable parameters in the model.
- (b) If the entire model is stored using INT8 quantization (1 byte per parameter), how much memory (in GB) is required to store the model?

Question 2: Counting number of parameters (solution)

(a) Trainable parameters.

Vocabulary = (60,000) (1024) (Assume same is used for unembedding)

$d_{ff} = 2d$.

$$\text{Encoder} = \left(h \times \frac{d}{h} \times d \right) \times 3 + \left(\frac{d}{h} \right) (h)(d) + (d)(2d)(2)$$

$\downarrow$
 w_k, w_q, w_v

$\downarrow$
 w_o

$\downarrow$
Feed forward

$$\begin{bmatrix} 2d & 2d \\ 2d & 2d \end{bmatrix} = \begin{matrix} 3d^2 + d^2 + 4d^2 \\ 8d^2 \end{matrix} \quad (d)$$

$$\text{Decoder} = \underbrace{2(4d^2)}_{2 \times \text{attention}} + \underbrace{4d^2}_{\text{feed forward}} = 12d^2 \quad (c)$$

$$\# \text{ parameters} = (8d^2 L_1) + (12d^2 L_2) + (60k)(1024)$$

$$\begin{matrix} \downarrow & \downarrow \\ L_1 = 24 & L_2 = 12 \\ d = 1024 \end{matrix}$$

Question 3: Counting number of parameters (Multi-head with shared K/V)

Now consider the same model setup as in Question 2. However, you apply the following variation in every attention module: a single shared Key projection and a single shared Value projection are used across all 16 heads, while maintaining 16 separate Query projections (one per head).

- (a) Recalculate the total number of trainable parameters under this modification.
- (b) What is the total reduction in parameters compared to Question 2?

Question 3: Counting number of parameters (Multi-head with shared K/V) (solutions)

Variation: Single key & values across all 16 attⁿ heads

params in self attⁿ $\Rightarrow$ $\frac{16 \cdot w_q}{d^2}$, $\frac{1 \cdot w_k}{\frac{d^2}{16}}$, $\frac{1 \cdot w_v}{\frac{d^2}{16}}$, $\frac{w_o}{d^2}$

Total/self attⁿ = $2d^2 + \frac{d^2}{8} = \frac{17}{8} d^2$ [and not $1d^2$]

Chage from per. overall $\frac{15}{8} d^2$ per attⁿ [self-encoder
self-decoder
cross

$\Rightarrow \frac{\frac{15}{8} d^2 \times 24}{\Delta \text{ Encoder}} + \frac{\frac{15}{8} d^2 \times 2 \times 12}{\Delta \text{ Decoder}} \approx 94.37m$

Question 4: Sampling strategies

Suppose a language model produces the following logits for the next-token prediction over the vocabulary $\{w_1, w_2, w_3, w_4\}$:

$$[2.5, 1.0, 0.5, -1.0]$$

At inference time, you apply different decoding strategies. Clearly justify your answer by computing the probabilities.

- ① If you use **greedy decoding**, which token will be selected?
- ② If you use **random sampling with temperature** $T = 0.5$, which token(s) are most likely to be selected? Suppose T_1 is used for code generation and T_2 is used for creative writing. What is the relationship between T_1 and T_2 ($T_1 < T_2$, $T_1 = T_2$, or $T_1 > T_2$)? Briefly justify.
- ③ If you use **Top- k sampling** with $k = 2$, which tokens remain eligible for selection?
- ④ If you use **Top- p sampling** with $p = 0.7$, which tokens will be included?
- ⑤ Which of the above decoding methods are **deterministic**?

Question 5: Parameter-Efficient Fine-Tuning

Suppose you want to use a T5 model with model dimensionality $d_{\text{model}} = 1024$ and 24 layers for three tasks: (1) text summarization, (2) question answering, and (3) text simplification. The base T5 parameters remain frozen. You adapt the model separately for each task using one of the following methods:

- **Prompt Tuning:** A soft prompt of length 10 is learned for each task.
- **Prefix Tuning:** A prefix of length 10 is learned per layer,
- **LoRA:** Applied to the Query and Key projection matrices in every layer, with rank $r = 8$.
- **Full Fine Tuning**

Assume that all prompt and prefix vectors have dimension 1024.

Calculate the number of parameters that must be stored at inference time.

Question 5: Parameter-Efficient Fine-Tuning (Solution)

5) a) Prompt tuning (extra parameters)

$$= 3 \times 1024 \times 10$$

↓ ↘ ↗
#tasks d_{model} prompt length

b) Prefix tuning (extra parameters)

$$= 3 \times 1024 \times 10 \times 24$$

↓ ↓ ↘ ↗
#tasks d_{model} ~~prompt~~ length #Layers

Note that prefix is applied only at encoder layer.

$$(c) \left(\left((2d_o + r + d) \times 2 \right) \times 3 \right) \times 3 \times 24 \rightarrow \# \text{ layers}$$

↓ ↓ ↗
Query, 3 attentions 3 tasks
Key (2 in decoder +
 1 in encoder)

d) $28d^2L$

Question 6: AI Vulnerabilities

Consider the PAIR algorithm (Chao et al., 2023) taught in class.

- (a) Can early stopping be used to optimize the search efficiency? If so, how?
- (b) Suggest one heuristic that PAIR could use to prioritize promising prompts.
- (c) How could PAIR be used to strengthen AI defenses rather than just for red teaming? State only two points with clear justification.
- (d) PAIR costs \$0.03 per successful jailbreak on Vicuna, while GCG requires 256,000 queries per jailbreak. If PAIR is used to jailbreak 500 prompts, compute the total cost. If GCG's cost per query is \$0.00001, compute the total cost for the same 500 jailbreaks. Calculate how many times more expensive GCG is compared to PAIR.

Question 6: AI Vulnerabilities (Solution)

- **Early Stopping:** If a prompt achieves a **near-jailbreak score**, stop refining it.
- Use **confidence scores from past jailbreak attempts** to weight prompt selection.
- Prioritize prompts that **previously elicited borderline responses** from the LLM.

1. Bias Toward More Defensive Evaluations

- Modify the judge function to prioritize **false negatives** (i.e., flagging responses as unsafe even if they might be borderline acceptable).
- **Justification:** This conservatively prevents harmful outputs by erring on the side of caution, forcing the AI to adopt **safer and more neutral responses** in adversarial scenarios.

2. Context-Aware Risk Scoring

- Enhance the judge function by incorporating **dynamic contextual risk analysis** instead of static rule-based evaluations.
- **Justification:** By integrating **reinforcement learning** and historical attack data, the AI can **adaptively learn** from evolving threats, strengthening its ability to **detect nuanced adversarial tactics** before exploitation.

Question 6: AI Vulnerabilities (Solution)

PAIR costs \$0.03 per successful jailbreak on Vicuna, while GCG requires 256,000 queries per jailbreak.

Tasks:

1. If PAIR is used to jailbreak 500 prompts, compute the total cost.
2. If GCG's cost per query is \$0.00001, compute the total cost for the same 500 jailbreaks.
3. Calculate how many times more expensive GCG is compared to PAIR.

Solution:

1. **PAIR Total Cost:**

$$500 \times 0.03 = 15$$

PAIR costs \$15 for 500 jailbreaks.

2. **GCG Total Cost:**

$$500 \times 256000 \times 0.00001 = 1280$$

GCG costs \$1,280 for 500 jailbreaks.

3. **Relative Cost Increase:**

$$\frac{1280}{15} \approx 85.33$$

GCG is 85x more expensive than PAIR.